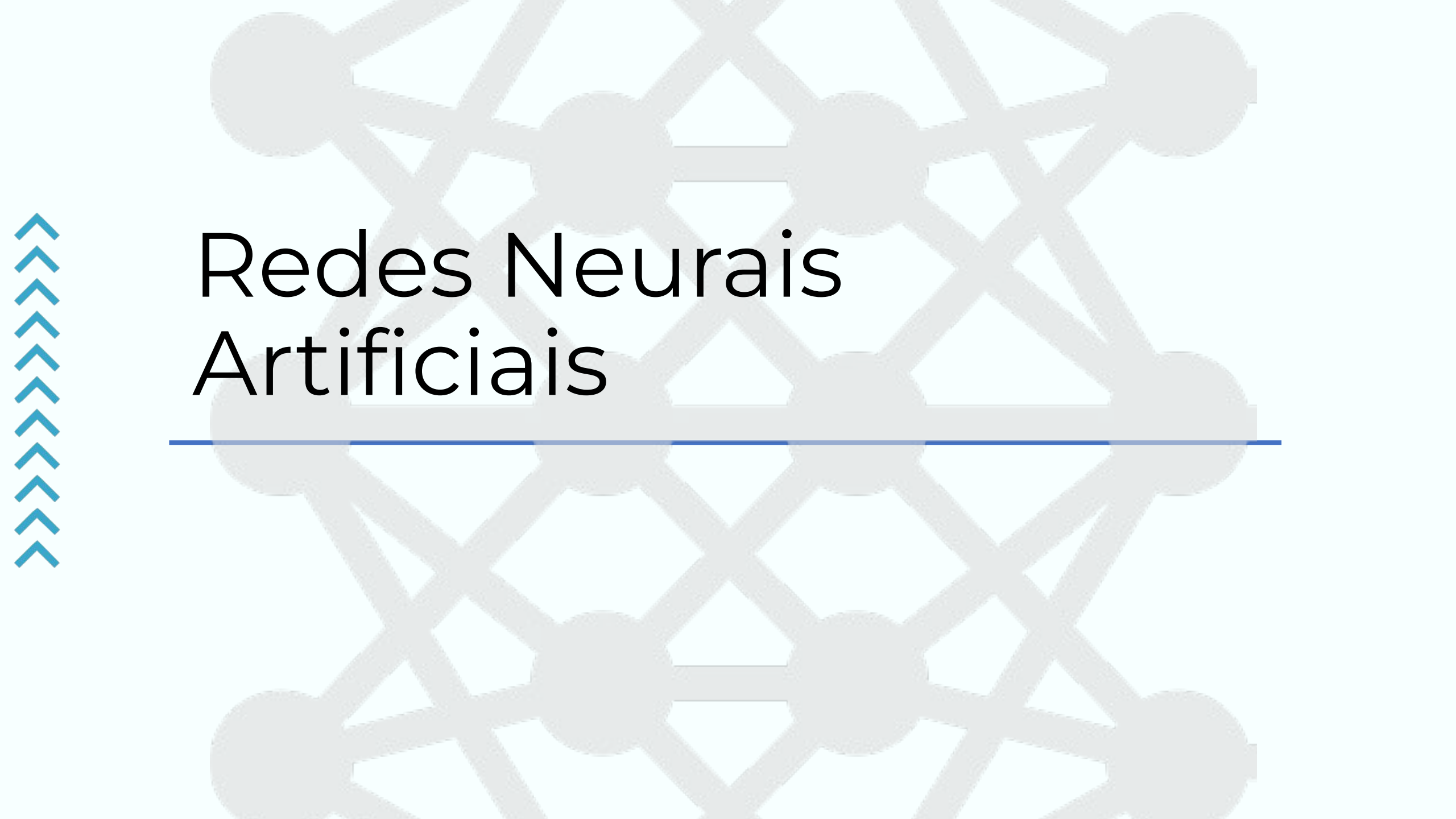




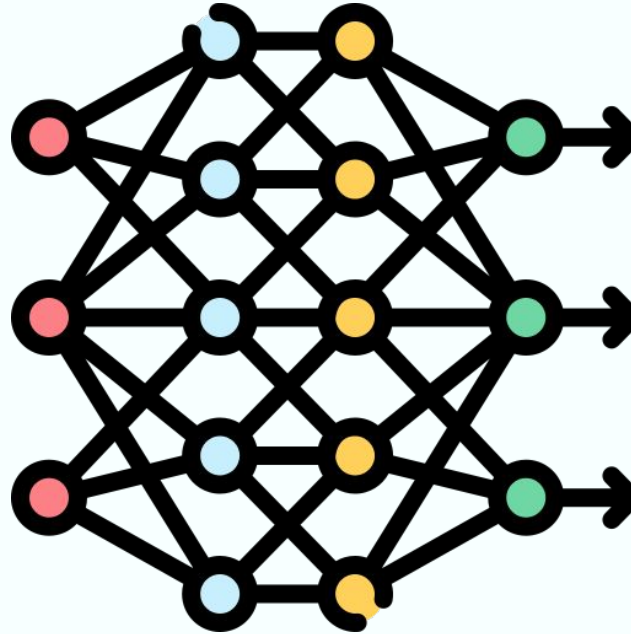
Redes Neurais Artificiais

Tópicos

Redes Neurais Artificiais

- **Introdução às Redes Neurais Artificiais**
 - Definição
 - Evolução histórica
 - Redes Neurais Feedforward
 - Vantagens e desvantagens das Redes Neurais
 - Algoritmo backpropagation
 - Funções de ativação e funções de custo mais comuns
- **Aplicação de Redes Neurais com Python**
 - Etapas para treinar uma rede neural
 - *Hands-on Lab*
- **Referências Bibliográficas**

- **Rede Neural Artificial (RNA)** – é um modelo computacional inspirado na estrutura e no funcionamento do cérebro humano. Ela é composta por unidades interconectadas chamadas neurônios artificiais ou nós, que trabalham em conjunto para processar informações e resolver problemas complexos.



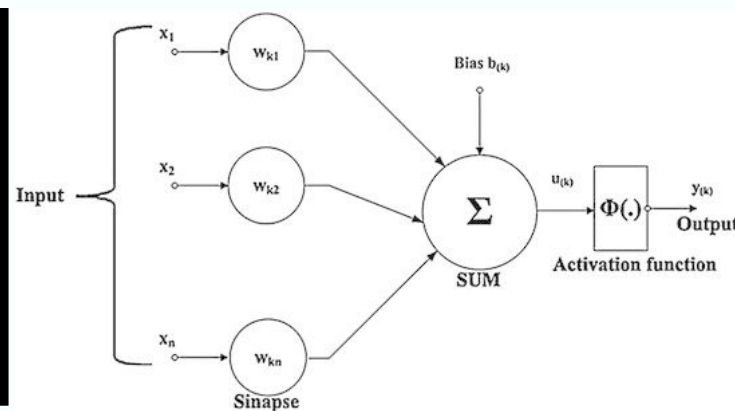
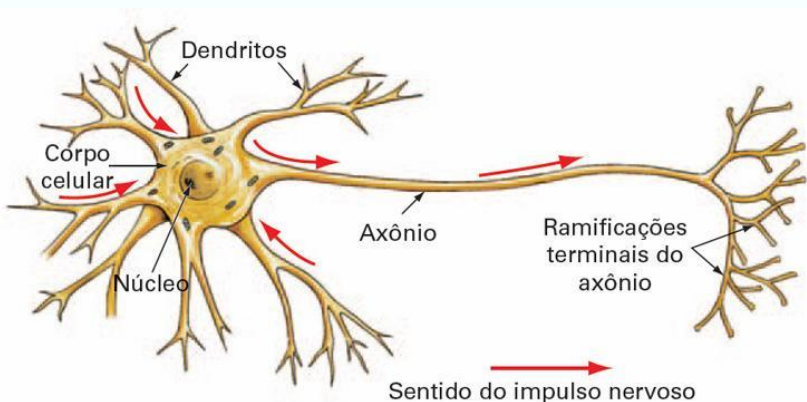
Introdução às Redes Neurais Artificiais

Definição

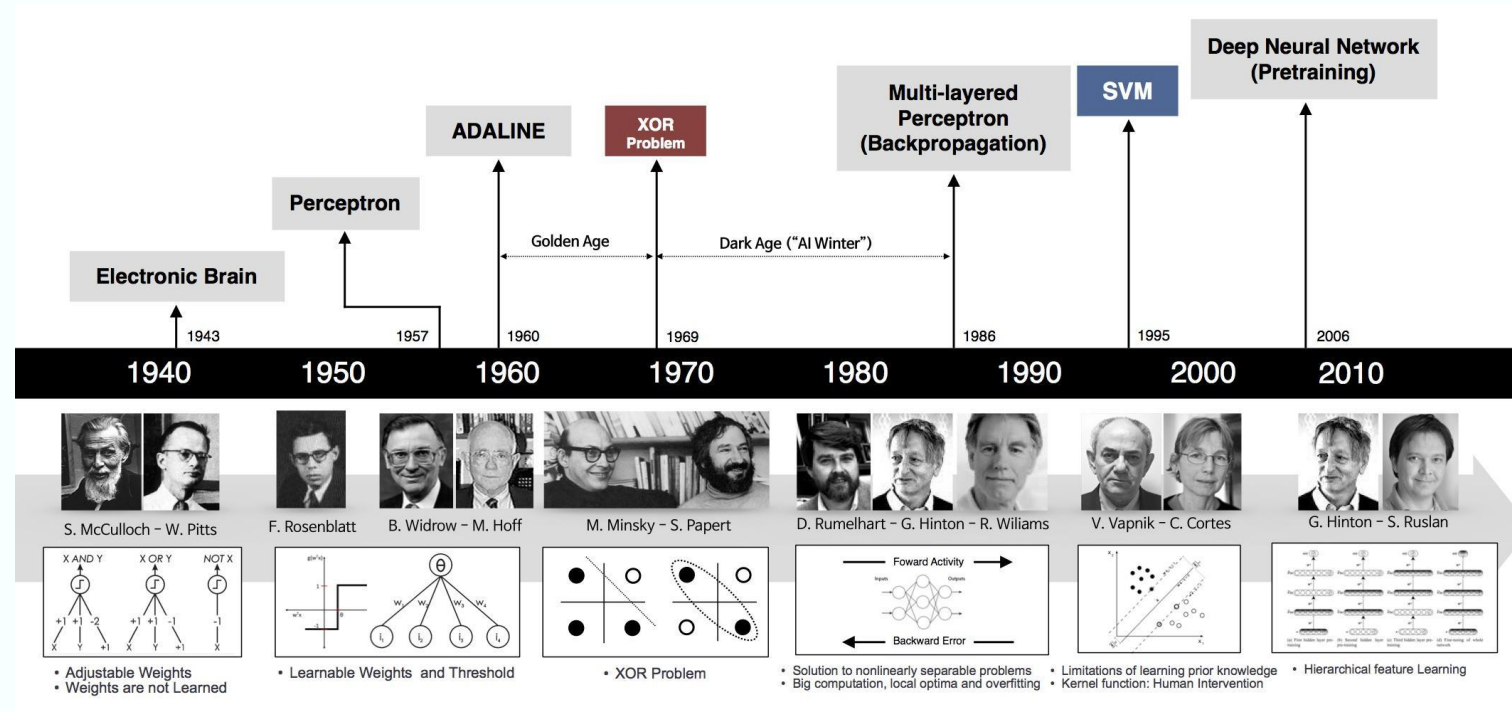
Neurônio biológico

VS

Neurônio matemático



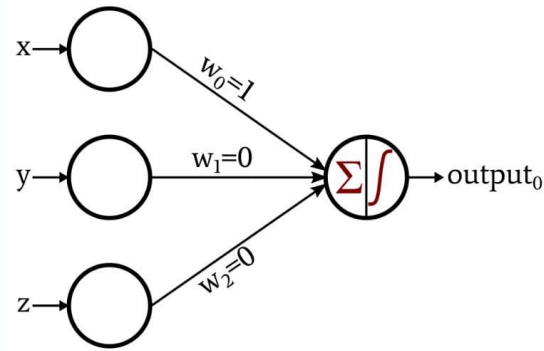
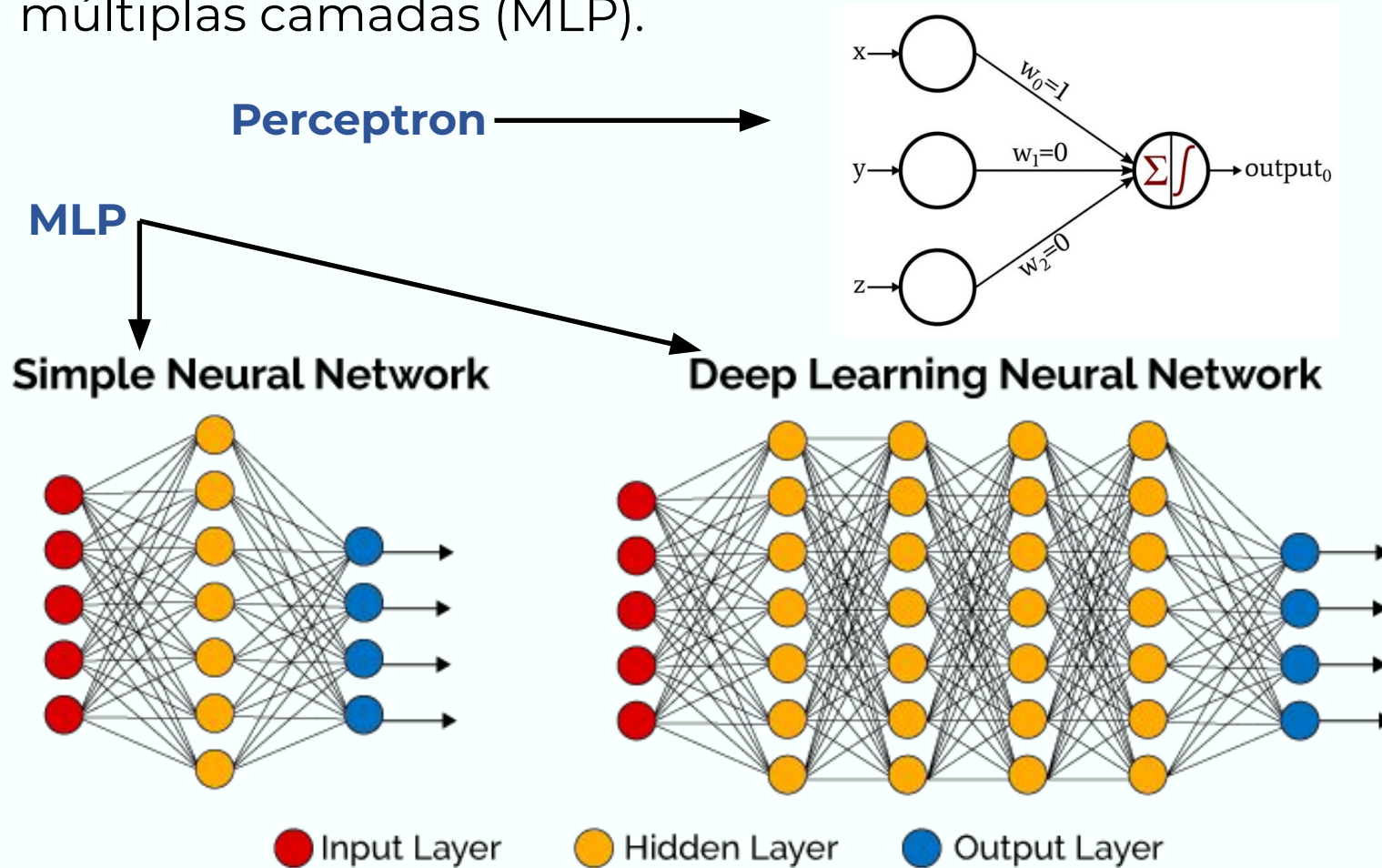
- Neurônio eletrônico
- Perceptron
- ADALINE
- MLP
- SVM
- Redes Neurais Profundas



Introdução às Redes Neurais Artificiais

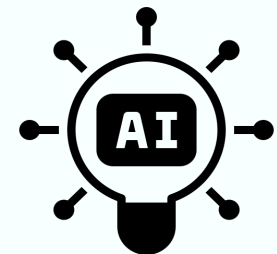
Evolução histórica

- **Redes Neurais Feedforward (FFNN)** – são um tipo de rede neural onde a informação flui em uma única direção: **Entrada** -> **Oculto(s)** (se houver) -> **Saída**
- As FFNN incluem: perceptron e perceptron de múltiplas camadas (MLP).

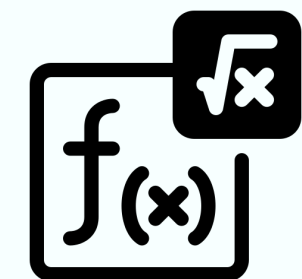


Introdução às Redes Neurais Artificiais

Redes Neurais Feedforward (FFNN)



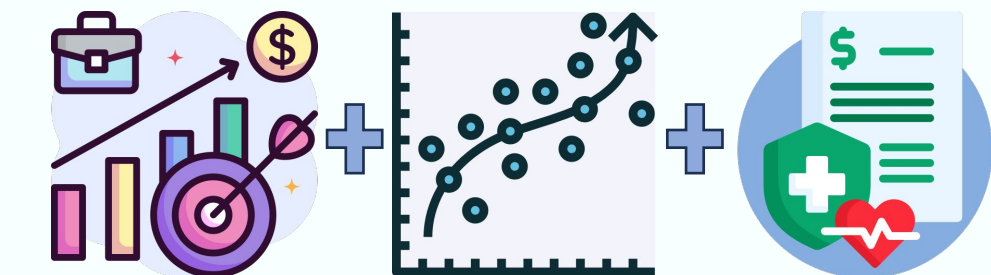
Capacidade de Generalização – quando bem treinadas conseguem generalizar bem para novos dados não vistos.



Capacidade de Modelar Relações Complexas – resolve problemas que não são linearmente separáveis.



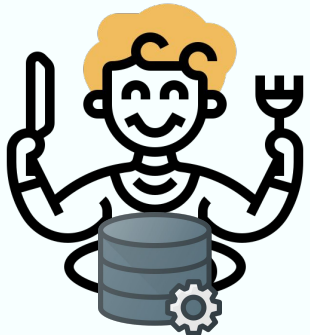
Escalabilidade – ideal para trabalhar com grandes volumes de dados (Big Data).



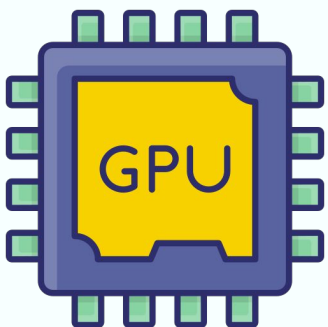
Versatilidade – aplicado em várias áreas.

Introdução às Redes Neurais Artificiais

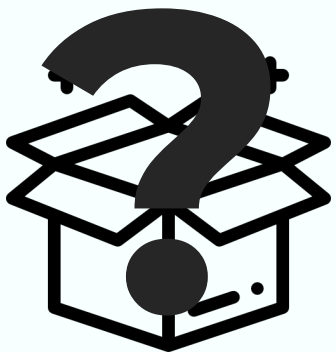
Vantagens das Redes Neurais



Data greedy – Necessidade de Grandes Conjuntos de Dados para treinamento



Treinamento Computacionalmente Intensivo (exigência de GPU e outros recursos de hardware)



Interpretação Difícil – as decisões tomadas nem sempre são compreendidas, por esse motivo chamam de caixas-pretas.



Sensibilidade a Dados de Treinamento – dados ruins conduzem a resultados ruins.

Introdução às Redes Neurais Artificiais

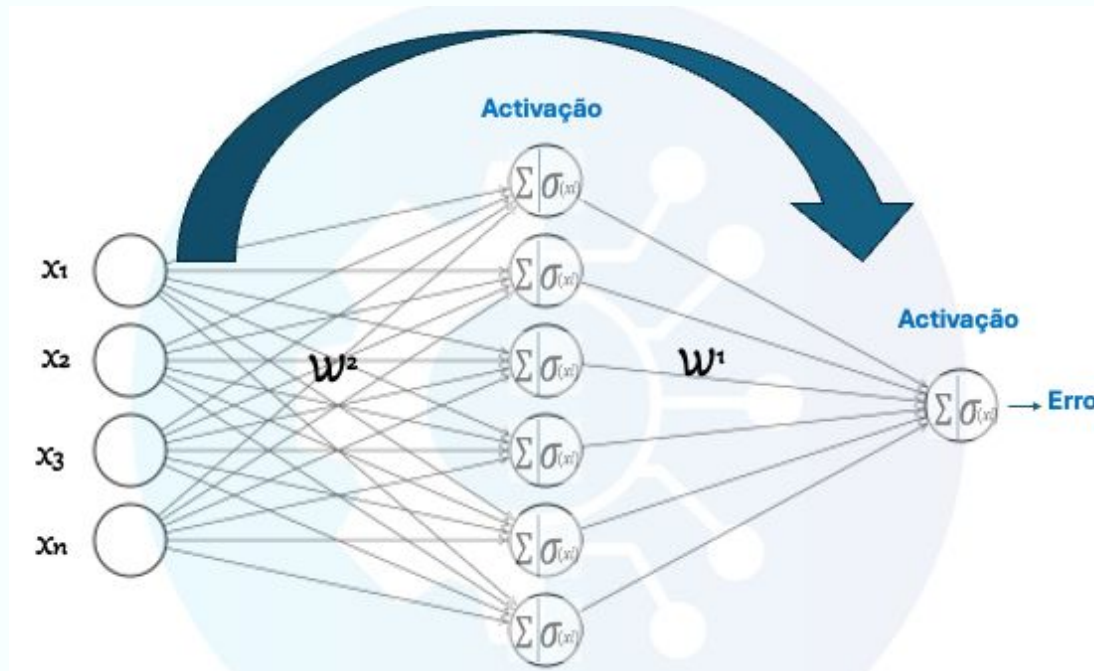
Desvantagens das Redes Neurais



1. Inicialização dos Pesos – os pesos da rede são inicializados com valores pequenos e aleatórios.

2. Feedforward (Propagação Direta) – as entradas do treinamento são passadas pela rede camada por camada. Cada neurônio calcula sua saída aplicando a função de ativação à soma ponderada de suas entradas. As saídas da última camada são as previsões do modelo.

3. Cálculo da Função de Custo – a função de custo é calculada com base na diferença entre as previsões da rede e os valores reais (ou desejados).



Introdução às Redes Neurais Artificiais

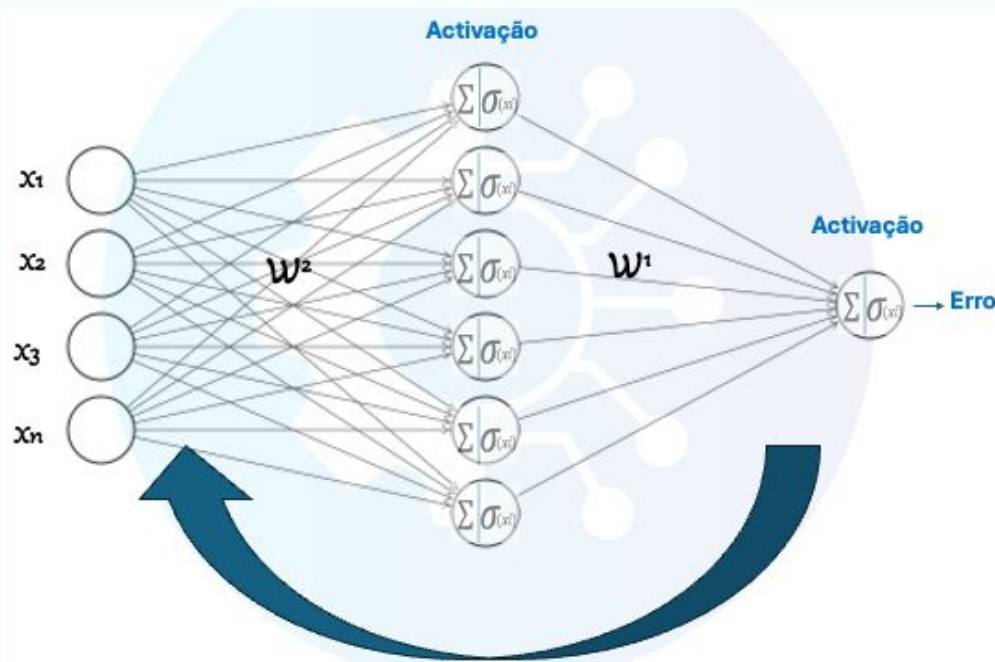
Algoritmo
Backpropagation

4. Backpropagation (Propagação do Erro)

- **Cálculo do Erro na Saída:** Para cada neurônio na camada de saída, calcula-se o erro comparando a saída prevista com o valor real. O erro da camada de saída é calculado com a derivada da função de custo em relação à saída da rede.

- **Propagação do Erro para Camadas Anteriores:** O erro é propagado para trás, da camada de saída até as camadas ocultas, usando as derivadas das funções de ativação. Isso envolve calcular o gradiente do erro em relação a cada peso da rede.

5. Atualização dos Pesos – os pesos são ajustados na direção oposta ao gradiente do erro, minimizando a função de custo.

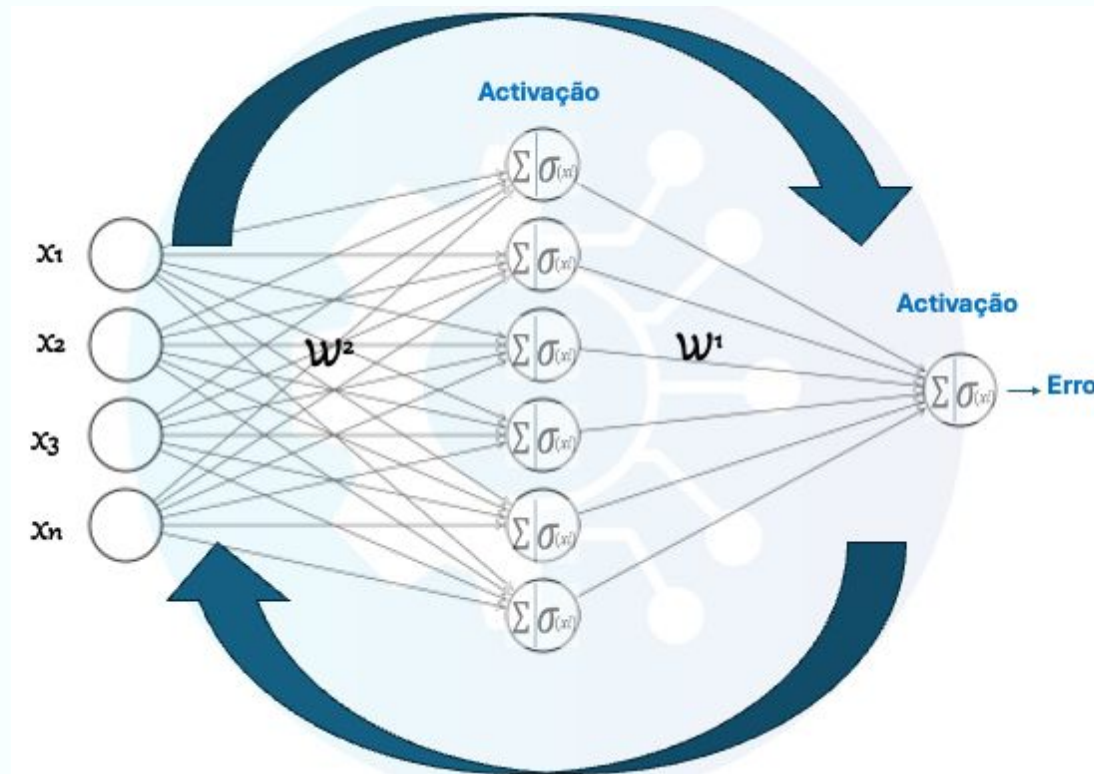


Introdução às Redes Neurais Artificiais

Algoritmo
Backpropagation



6. Repetição – os passos 2 a 5 são repetidos para cada amostra ou lote de amostras até que a função de custo convirja (isto é, pare de diminuir significativamente). Esse processo é realizado para múltiplas iterações (épocas) até que o modelo esteja adequadamente treinado.



Introdução às Redes Neurais Artificiais

Algoritmo
Backpropagation

- **ReLU** (Rectified Linear Unit): Função de ativação não-linear mais popular.

$$f(x) = \max(0, x)$$

- **Sigmoid**: Comprime os valores entre 0 e 1, usada em problemas de classificação.

$$f(x) = \frac{1}{1 + e^{-x}}$$

- **Tanh**: Comprime os valores entre -1 e 1, usada em modelos onde a saída pode ter sinais negativos.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **Softmax**: usada em modelos de classificação multiclases. Converte um vetor de valores em probabilidades, onde a soma de todas as probabilidades é igual a 1.

$$z_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

Introdução às Redes Neurais Artificiais

Funções de ativação comuns em Redes Neurais



- **Mean Squared Error (MSE)** – usado, geralmente, em problemas de regressão.

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Onde:

y_i - é o valor real.

$\hat{y}_i$ - é o valor previsto pelo modelo.

n - é o número de amostras.

- **Mean Absolute Error (MAE)** – usado, geralmente, em problemas de regressão.

$$\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Onde:

y_i - é o valor real.

$\hat{y}_i$ - é o valor previsto pelo modelo.

n - é o número de amostras.

Introdução às Redes Neurais Artificiais

Funções de custo comuns em Redes Neurais



- **Cross-Entropy** – Usado, geralmente, em problemas de classificação binária.

$$-\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Onde:

- $y_{i,c}$ - é o valor binário (0 ou 1) indicando se a classe C é a classe correta para amostra i.
- $\hat{y}_{i,c}$ - é a probabilidade prevista pelo modelo para a classe C .
- C - é o número total de classes.

- **Categorical Cross-Entropy** – Usado, geralmente, em problemas de classificação multiclasse.

$$-\sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Onde:

- $y_{i,c}$ - é o valor binário (0 ou 1) indicando se a classe C é a classe correta para amostra i.
- $\hat{y}_{i,c}$ - é a probabilidade prevista pelo modelo para a classe C.
- C - é o número total de classes.

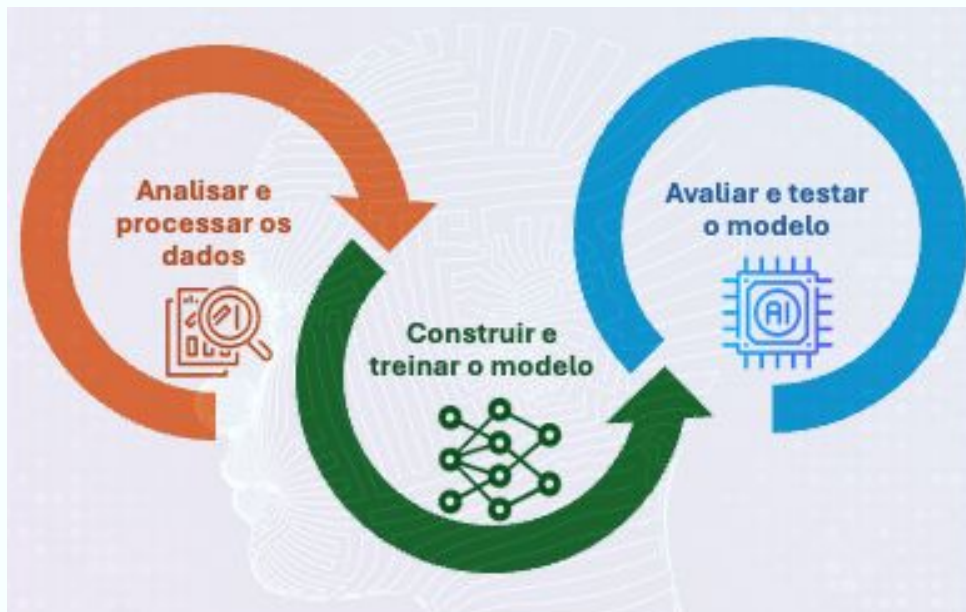
Introdução às Redes Neurais Artificiais

Funções de custo comuns em Redes Neurais



Entre várias tarefas, o pré processamento envolve: tratar valores faltantes e outliers; codificar (vetorizar) os dados; normalizar os dados; separar os dados (treino, validação e teste); etc.

1



2

A construção do modelo envolve: definir a topologia (camadas, nr de neurônios, funções de ativação); definir a configuração (função de custo, tamanho do lote de treinamento, taxa de aprendizado, número de épocas, critério de parada; etc.

A análise consiste em calcular as métricas; plotar gráficos do treinamento; ajustar parâmetros e re-treinar o modelo (se necessário)

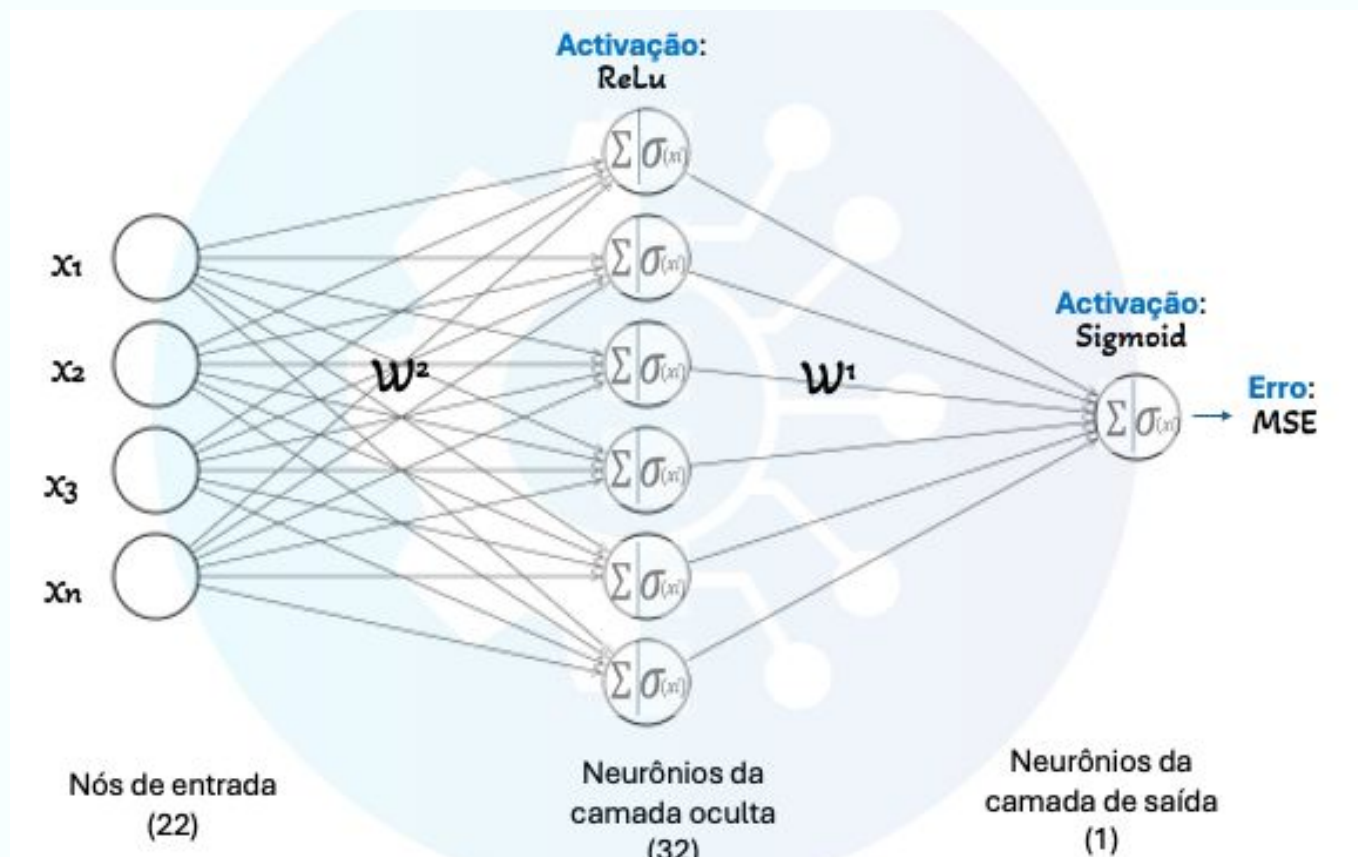
3

Aplicação de Redes Neurais com Python

Etapas para treinar uma rede neural no python



Objetivo: Implementar uma Rede Neural com Keras (biblioteca do python para Deep Learning) com as características da imagem abaixo



[Link para o Notebook no Google Colab](#)

Aplicação de Redes Neurais com Python

Hands-On Lab

• Referências

- Chollet, F., et al.: Keras. <https://keras.io> (2015)
- Data Science Academy, Deep Learning book, <https://www.deeplearningbook.com.br>
- Géron, A.. Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow. " O'Reilly Media, Inc.", 2022.

Referências

